

Date. ____/____/____

DATA STRUCTURES & ALGORITHM

- By JAYA

BEGINNER → ADVANCE
GUIDE

ABHISHEK RATHOR

INSTAGRAM - @Syntax Error

- code.abhi07



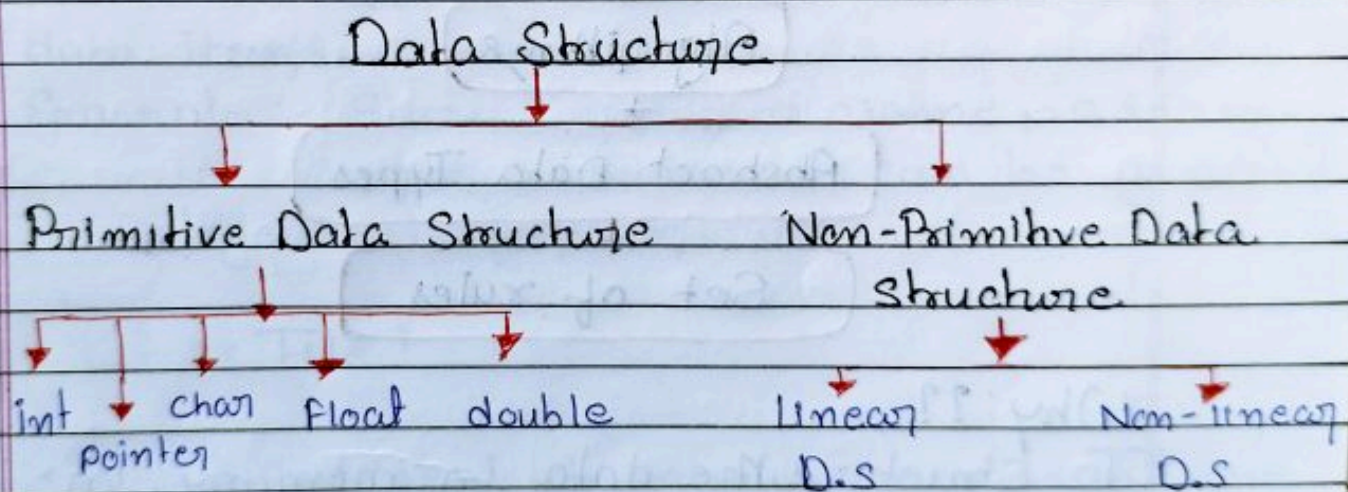
INDEX

Sr NO	TITLE OF TOPIC	PAGE NO.
1)	Data Structure Introduction	
2)	Classification of Data Structure	
3)	Introduction to algorithm	
4)	Asymptotic Notation	
5)	Basic of JAVA	
6)	Arrays	
7)	Strings	
8)	Searching & Binary Search	
9)	Recursion & Backtracking	
10)	Hashing	
11)	Stack & Queue	
12)	Linked list.	



What is DATA STRUCTURE ?

- Data Structure is a way to store and organize data so that it can be used Efficiently.
- As per name indicates itself that organizing the data in memory.
- The data Structure is not any programming language like C, C++, Java etc. It is set of algorithms that we can use in any programming language to structure data in memory.



Linear Data Structure :

- The arrangement of Data in the sequential manner is known as linear Data Structure.
- The data Structure used for this purpose are **Arrays, Linked-list, Stacks and queues.**
- In this Data Structure, one element is connected to only one another element in a linear form.

Date. —/—/—

Algorithms and Data Structures



Non-linear Data Structure :-

→ When one element is connected to the 'n' number of elements known as non-linear Data Structure.

Example :- Trees and Graphs

→ In this case, elements are arranged in a random manner.

Algorithms and Abstract Data Types ??

Algorithms



Abstract Data Types



Set of rules

Why ??

To structure the data in memory, 'n' number of algorithms are proposed, and all these algorithms are known as **Abstract Data Types**.

An Abstract Data Type tells **what is to be done** and data structure tells **how is to be done**?



ADT Gives us the blueprint while data structure provides the implementation part.

What is **Data**?

- Data can be defined as the elementary value/ Collection of values.
- For Examples :- Student's name and its id are the data about student.

What is **Record**?

- Record can be defined as collection of various data items.
- Example :- Student entity → name, address, course and marks can be grouped together to form record.

What is **File**?

- File is a collection of various records of one type of entity.
- Example :- If there are 60 employees in class, then there will 20 records in related file where record contains info of employees.

What is **Attribute and Entity**?

An entity represents class of certain objects. it contains various attribute each attribute



represents particular property of that entity

What is need of Data Structure?

As applications are getting complexed and amount of data is increasing day by day, there may arise following problems:-

- ⇒ **Processor Speed:-** As data is growing day by day to the billions of files per entity, processor may fail to deal with that amount of data.
- ⇒ **Data Structure:-** Consider an inventory size of 100 items in store, if our application needs to search for a particular item, it needs to traverse 100 items every time, results in slowing down process.
- ⇒ **Multiple Requests:-** If thousands of users are searching data simultaneously on a web server, then there are chances that to be failed to search during that process.

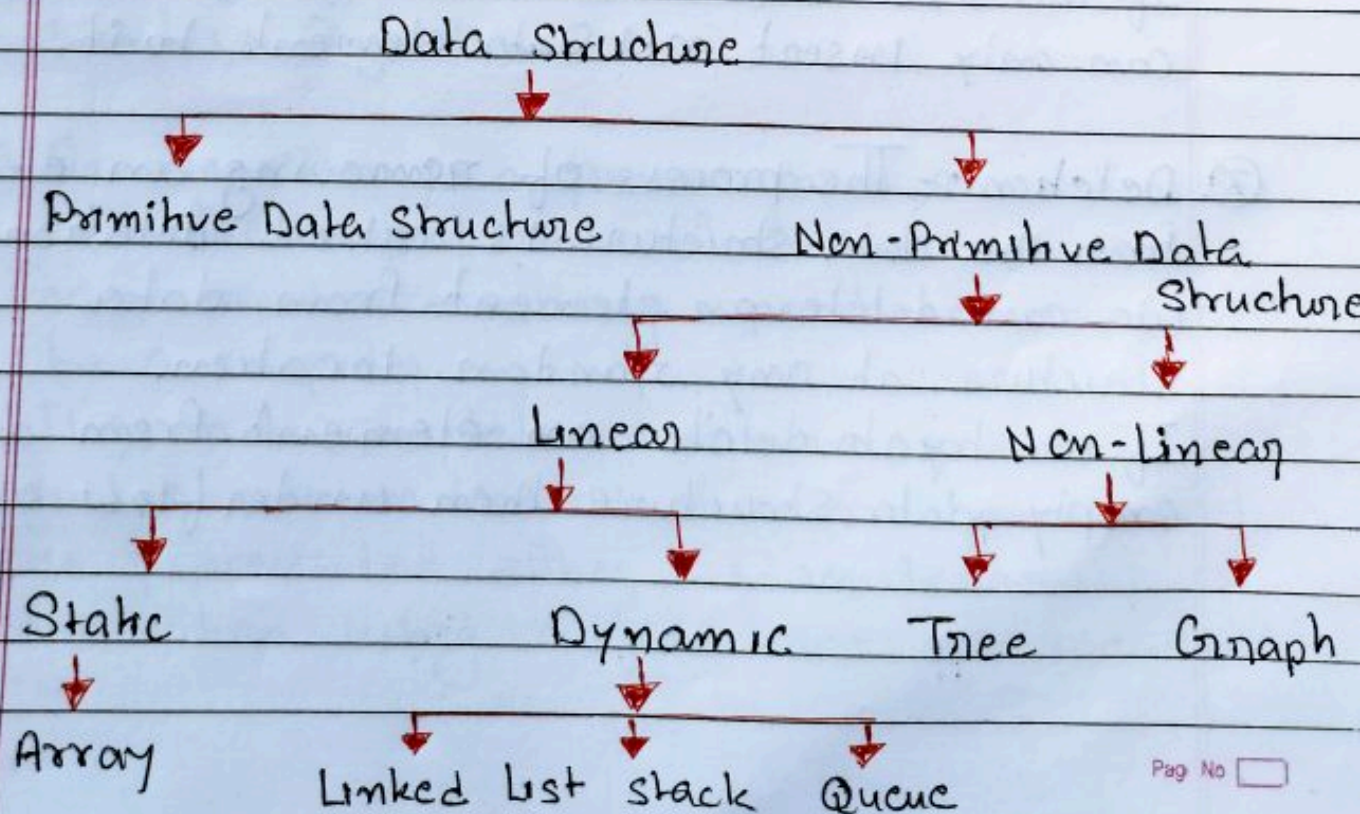
To solve these problems, data structures are used. Data is organized to form a data structure in such a way that all items are not required to be searched and require data can be searched instantly.



Advantages of DATA STRUCTURE :-

1. **Efficiency** :- If the choice of a data structure for implementing a particular ADT is proper, it makes program very efficient in terms of time and space.
2. **Reusability** :- The data structure provides reusability means that multiple client programs can use the data structure.
3. **Abstraction** :- The data structure specified by the ADT also provides level of abstraction. The client cannot see internal working of data structure, so it does not have to worry about implementation.

DATA STRUCTURE Classification :-



Operations on DATA STRUCTURE :-

- ① **Traversing** :- Every data structure contains a set of data elements. Traversing data structure means visiting each element of data structure in order to perform some specific operation like Searching or Sorting.

Example :- If we need to calculate average of marks obtained by a student in r different subject, we need to traverse compute array of marks and calculate total sum, then we will divide that sum by no. of subjects i.e. to find average.

- ② **Insertion** :- Insertion can be defined as the process of adding the elements to the data structure at any location.

If the size of data structure is n then we can only insert $n-1$ data elements to it.

- ③ **Deletion** :- The process of removing an element from the data structure is called deletion.

We can delete an element from data structure at any random location.

If we try to delete an element from an empty data structure then underflow occurs.



- ④ Searching :- The process of finding the location of an element within data structure is called searching. There are two algorithms to perform searching, linear search & Binary Search.
- ⑤ Sorting :- The process of arranging the data structure in a specific order is called as sorting. There are many algorithms that can be used to perform sorting. For example :- Insertion Sort, Selection Sort, Bubble Sort etc.
- ⑥ Merging :- When two lists list A and list B of size m & n respectively of similar types of elements, clubbed or joined to produce third list, list C of size $(m+n)$, then this process is called merging.

Data Structure And Algorithm

What is Algorithm?

- An algorithm is a process or a set of rules required to perform calculations or some other problem-solving operations especially by a computer.
- It is not complete program or code, it is just a solution (logic) of a problem, which can be represented either as an informal description using a flowchart or pseudocode.

Characteristics of an Algorithm

- **Input:-** An algorithm has some input values. We can pass 0 or some input value to an algorithm.
- **Output:-** We will get 1 or more output at end of an algorithm.
- **Unambiguity:-** An algorithm should be unambiguous which means that instruction in an algorithm should be clear and simple.
- **Finiteness:-** An algorithm should be have finiteness means limited number of instructions.
- **Effectiveness:-** An algorithm should have finite as each instruction in an algorithm affects the overall process.

Approaches in Algorithm:-

- 1) **Brute force Algorithm:-** The general logic structure is applied to design an algorithm. It is also known as exhaustive search algorithm that searches all possible to provide required solution.



Such algorithms have two types :-

- 1) Optimizing :- Finding all solutions of a problem and then take out the best solution is known that it will terminate if the best solution is known.
 - 2) Sacrificing :- As soon as the best solution is found, then it will stop.
- ⇒ Divide and conquer :- This breaks down the algorithm to solve the problem in different methods. It allows you to break down problem into different methods, and valid output is produced for the valid input. This valid output is passed to some other function.
- ⇒ Greedy Algorithm :- It is an algorithm paradigm that makes an optimal choice in each iteration with the hope of getting best solution. It is easy to implement and has faster execution time. But there are very rare cases in which it provides the optimal solution.

The major categories of **algorithms** are given below:-

- Sort :- Algorithm developed for sorting the items in a certain order.
- Search :- Algorithm developed for searching the items inside a data structure.

- **Delete:-** Algorithm developed for deleting the existing element from the data structure.
- **Insert:-** Algorithm developed for insert an item inside a data structure.
- **Update:-** Algorithm developed for updating the existing element inside a data structure.
- **Algorithm Analysis:-**
The algorithm can be analyzed in two levels i.e. first is before creating the algorithm and second is after creating the algorithm.

There are two analysis of an algorithm.

- **Priori Analysis:-** Here, priori analysis is the theoretical analysis of an algorithm which is done before implementing the algorithm.
- **Posterior Analysis:-** Here, posterior analysis is a practical analysis of an algorithm. The practical analysis is achieved by implementing algorithm using any programming language.

⇒ **Algorithm Complexity:-**

The performance of the algorithm can be measured in two factors:-

(a) Syntax Error
— Abhishek Rathor
Date. ___/___/___



⇒ Time Complexity :- The time complexity of an algorithm is the amount of the time required to complete the execution. The time complexity of an algorithm is denoted by the big O notation.

Here big O Notation is the asymptotic notation to represent time complexity. The time complexity is mainly calculated by counting the number of steps to finish execution.

```
Sum = 0; // Suppose we have to calculate the sum of n
For i = 1 to n                      numbers
Sum = Sum + i; // when the loop ends then sum holds the
return sum;                      sum of n numbers
```

In the above code, the time complexity of the loop statement will be at least n , and if value of n increases, then time complexity also increases.

We generally consider the worst-time complexity as it is maximum time taken for any given input size.

⇒ Space Complexity :- An algorithm's space complexity is the amount of space required to solve a problem and produce an output. Similar to the time complexity, space complexity is also expressed in big O notation.

Space Complexity = Auxiliary Space + Input Size

The following are the types of algorithms:-

1. Search Algorithm:- On each day, we search for something in our day to day life.

Similarly, with the case of computer, huge data is stored in a computer that whenever user asks for any data then the memory and computer searches for that data in the memory and provides that data to the user.

There are mainly two techniques available to search data in an array:

- Linear Search
- Binary Search

2. Sorting Algorithm:- Sorting Algorithms are used to rearrange elements in an array or a given data structure either in an ascending or descending order.

The comparison operator decides the new order of the elements.

SYNTAX ERROR | Abhishek Rathor

These handwritten notes are exclusively created for educational purposes under the SYNTAX ERROR brand.

Unauthorized use, duplication, or redistribution in any form is strictly prohibited

for collaboration or queries: @code.abhis07



Asymptotic Analysis:- The method of determining the running time of an algorithm in terms of input size. It helps compare the efficiency of different algorithms as the input size grows large.

⇒ Worst case:- Maximum time taken by an algorithm for any input.

⇒ Average case:- Expected time over all inputs
Best case:- Minimum time the algorithm takes on any input.

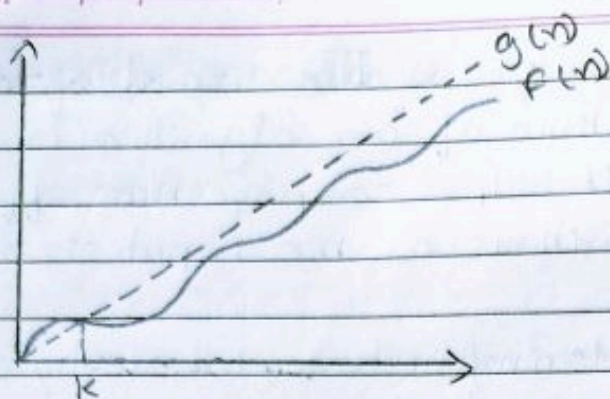
⇒ Asymptotic analysis uses notations to describe the running time in a way that ignores constant factors and lower-order terms.

1. Big O Notation (O)

- Describes the upper bound of the running time of an algorithm.
- Represents the worst-case time complexity
- If a function $f(n)$ and $g(n)$ satisfy:

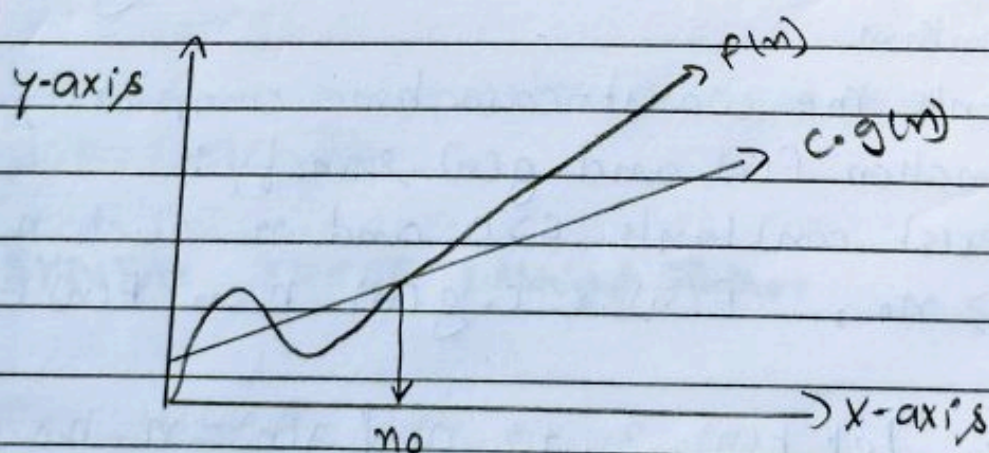
There exist constants $c > 0$ and n_0 such that for all $n \geq n_0$, $f(n) \leq c \cdot g(n)$ then $f(n) \in O(g(n))$

• Example: Let $f(n) = 3n + 2$ and $g(n) = n$, then $f(n) = O(n)$ since $f(n)$ grows linearly like n .



2. Omega Notation $\Omega(g(n))$

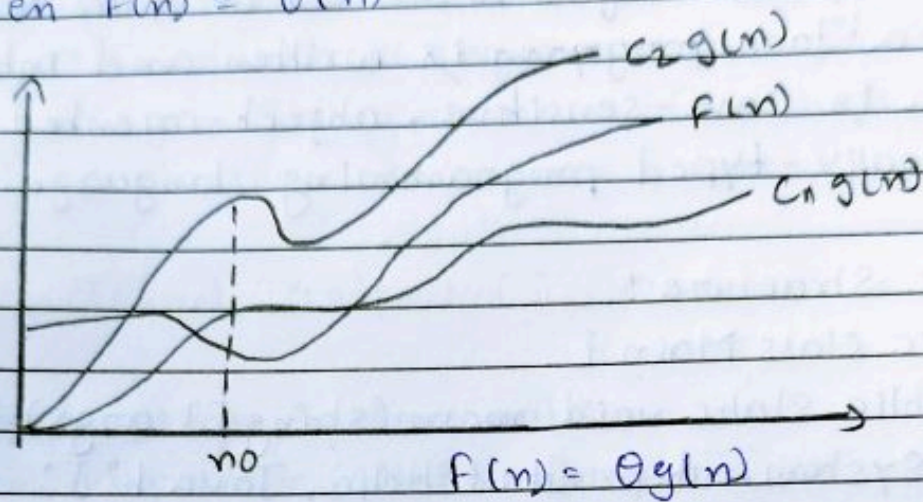
- Describe the best-case performance
- It gives a lower bound - the minimum time an algorithm will take
- Let $f(n)$ and $g(n)$ be functions of n where n is steps required to execute programs $f(n) = \Omega(g(n))$
There exist constant $c > 0$ and n_0 such that for all $n \geq n_0$, $f(n) \geq c \cdot g(n)$
- Example:- Let $f(n) = 3n + 2$ and $g(n) = n$, then $f(n) = \Omega(n)$ because it grows at least like n



3. Theta Notation (Θ)

- Describe the average-case or tight bound
- It means the algorithm grows exactly at the same rate as $g(n)$ for large n

- If $F(n) = \Theta(g(n))$, then there are constants C_1 and C_2 such that $C_1 \cdot g(n) \leq F(n) \leq C_2 \cdot g(n)$
- Example: If $F(n) = 3n+2$ and $g(n) = n$, then $F(n) = \Theta(n)$



Basics of Java

⇒ Java Syntax

- Java Syntax refers to the set of rules that define how a Java program is written and interpreted.
- Java is case-sensitive, object-oriented, statically-typed programming language.

Basic Structure:

```
public class Main {  
    public static void main (String [] args) {  
        System.out.println ("Hello, Java !");  
    }  
}
```

- Semicolon ';' : Every statement ends with a semicolon.
- Comments :
// This is a single-line comment
/* This is a multi-line comment */

⇒ Java Data Types

Java has two types of data types:

A. Primitive Data Types:

byte: 1 byte (e.g., byte a = 100;)
short: 2 bytes (e.g., short b = 10000;)
int : 4 bytes (e.g., int c = 123456;)
long : 8 bytes (e.g., long d = 123456789L;)
float : 4 bytes (e.g., float e = 5.75f;)

double : 8 bytes (e.g., double f = 19.99 ;)

char : 2 bytes (e.g., char g = 'A';)

boolean : 1 bit (e.g., boolean is Java = true)

B. Non-Primitive Data Types:

String: Represents a sequence of characters
(e.g., String name = "Abhishek";)

Array, classes, Interface, etc

=> Java Input/Output (I/O)

A. Output using System.out:

```
System.out.println("Hello, World!");
```

```
System.out.println("No new line");
```

```
System.out.printf("Value of x : %d", 10);
```

B. Input using Scanner class:

Steps:

1. Import java.util.Scanner;

2. Create Scanner object: Scanner sc = new Scanner
(System.in);

3. Use methods to read input: sc.nextInt(), sc.nextLine()
etc.

Example: import java.util.Scanner;

```
public class Main {
```

```
    public static void main (String[] args) {
```

```
        Scanner sc = new Scanner (System.in);
```

```
        System.out.print("Enter your name: ");
```



```

String name = sc.nextLine();
System.out.print("Enter your age:");
int age = sc.nextInt();
System.out.println("Hello " + name + ", you are" +
    age + " years old.");
}
}

```

Common Scanner Methods:

next(), nextLine(), nextInt(), nextDouble(),
nextBoolean()

Scanner Methods

Method	Description
next()	Reads a single word
nextLine()	Reads entire line
nextInt()	Reads integer
nextDouble()	Reads double
nextBoolean()	Reads boolean

Note on Input Issues

After nextInt(), if you use nextLine(), it may skip because nextInt() doesn't consume the newline. Use sc.nextLine() once after numeric inputs to avoid this.



Java Loops (for, while, do-while)

1. Introduction to Loops

Loops in Java are used to execute a block of code repeatedly until a specified condition is met.

2. For Loop

Syntax:

```
for(initialization; condition; update) {  
    // code to execute  
}
```

Example:

```
public class ForLoopExample {  
    public static void main (String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println ("i = " + i);  
        }  
    }  
}
```

Explanation:

Starts with $i = 1$

Runs while $i \leq 5$

Prints value of i and increments i in each iteration

3. While Loop

Syntax:

```
while (condition) {  
    // code to execute  
}
```


Example:

```

public class WhileLoopExample {
    public static void main (String[] args) {
        int i = 1;
        while (i <= 5) {
            System.out.println ("i = " + i);
            i++;
        }
    }
}

```

Explanation:

Initializes $i=1$ Prints value of i while $i \leq 5$

4. do-while Loop

Syntax:

do {

// Code to execute

} while (condition);

Example:

```

public class DowhileExample {
    public static void main (String[] args) {
        int i = 1;
        do {
            System.out.println ("i = " + i);
            i++;
        } while (i <= 5);
    }
}

```


Date. ___/___/___



Explanation:

Executes the code at least once

Then checks if $i \leq 5$ and continues

5. Special case: do-while Runs once

Example:

```
public class DOWHILEOnce {
```

```
    public static void main (String[] args) {
```

```
        int i=10;
```

```
        do {
```

```
            System.out.println ("Runs once even if condition  
                                is false");
```

```
        } while (i < 5);
```

```
    }
```

```
}
```

Output: Runs once even though condition is false initially

6. Summary Table

Loop type	Entry check	Runs at least once	Best when
a) For	Yes	No	Iterations known
b) while	Yes	No	Condition based Loop
c) do-while	No	Yes	Run once before checking

Date. ___/___/___

Java Conditional Statements (if-else, switch)

1. Introduction to Conditional Statements
Conditional Statements in Java allow the program to take decision based on conditions.

There are two primary types: if-else and switch

2. if-else Statement

Syntax:

```
if (condition) {  
    // code if condition is true  
} else {  
    // code if condition is false  
}
```

You can also use else-if for multiple conditions:

```
if (condition 1) {  
    // code block  
} else if (condition 2) {  
    // code block  
} else {  
    // default code  
}
```

Example:

```
public class IfElseExample {  
    public static void main (String[] args) {  
        int number = 10;  
        if (number > 0) {  
            System.out.println("Positive number")  
        }  
    }  
}
```


② Syntax Error

— Abhishek Rathor

Date. ___/___/___



```
} else {  
    System.out.println("Non-positive number");  
}  
}
```

Explanation:

checks if number is greater than 0

Prints the appropriate message based on the condition

3. Switch Statement

Used when you have multiple conditions based on the same variable.

Syntax:

```
Switch (expression){
```

```
    case value1:
```

```
        // code
```

```
        break;
```

```
    case value2:
```

```
        // code
```

```
        break;
```

```
    default:
```

```
        // default code
```

```
}
```

Example :

```
public class SwitchExample {
```

```
    public static void main (String[] args) {
```

```
        int day = 3;
```

```
        Switch (day) {
```

```
            case 1:
```

```
                System.out.println("Monday");
```


rehan khalid

Date. ___/___/___

```
break;  
case 2:  
    System.out.println("Tuesday");  
    break;  
case 3:  
    System.out.println("Wednesday");  
    break;  
default:  
    System.out.println("Invalid day");  
}  
}
```

Explanation:

Checks the value of 'day'

Executes the matching case and exits with break

If no case matches, runs the default case

4. Key Difference: if-else vs Switch

- If-else works with boolean expressions, while Switch compares a single variable to multiple values
- Switch is cleaner for multiple fixed values
- If-else is more flexible and supports ranges and complex conditions



Functions And Recursion in Java

⇒ A function (method) is a block of code that performs a specific task. It increases code reusability and improve readability.

⇒ Why use Functions?

Avoid code repetition

Enhances Modularity

Improves debugging and testing

Easy collaboration in large codebases

⇒ Syntax:-

```
<return_type> <function_name> (<parameters>) {  
    // Code block  
    return value;  
}
```

⇒ Example:

```
public static int add(int a, int b) {  
    return a+b;  
}
```

⇒ Types of Functions

1. Predefined (Standard Library) e.g., Math.max(), System.out.print()
2. User-defined Custom methods created by the programmer.

✍

⇒ Function Example

```
public class Main {
    public static int multiply(int a, int b) {
        return a * b;
    }

    public static void main(String[] args) {
        int result = multiply(5, 6);
        System.out.println("Multiplication is: " + result);
    }
}
```

- Time Complexity : $O(1)$
- Space Complexity : $O(1)$

⇒ Recursion :- A process where a method calls itself to solve smaller sub-problems of a main problem

Syntax:

```
<return_type> functionName(parameters) {
    if (base condition) {
        return result;
    }
    return functionName(modified parameters);
}
```

Example 1: Factorial

```
public class Factorial {
    public static int factorial(int n) {
        if (n == 0 || n == 1) return 1;
        return n * factorial(n - 1);
    }
}
```


Date. ___/___/___



```
public static void main (String[] args) {  
    System.out.println ("Factorial of 5 is : " +  
        Factorial (5));  
}
```

Dry Run : Factorial (3)

$$\begin{aligned}\text{Factorial}(3) &= 3 * \text{Factorial}(2) \\ &= 3 * 2 * \text{Factorial}(1) \\ &= 3 * 2 * 1 = 6\end{aligned}$$

Time Complexity : $O(n)$

Space Complexity : $O(n)$

Example 2 : Fibonacci

```
public class Fibonacci {  
    public static int fibonacci (int n) {  
        if (n == 0) return 0;  
        if (n == 1) return 1;  
        return fibonacci (n-1) + fibonacci (n-2);  
    }  
}
```

```
public static void main (String[] args) {  
    System.out.println ("Fibonacci of 6 is : " + fibonacci (6));  
}
```

Time Complexity : $O(2^n)$

Space Complexity : $O(n)$

Recursion Vs Iteration

Use function calls

Use loops

High Memory (stack)

Low memory

Slower due to overhead

Faster execution

⇒ When to Use Recursion

Divide-and-conquer algorithms (e.g., Merge Sort)

Tree traversal (DFS)

Backtracking (e.g., N-Queens)

Dynamic Programming (Use memoization)

⇒ Common Recursion Mistakes

Missing base case

Incorrect input modification

Incorrect return statements

VII Important Interview Questions

Factorial

Fibonacci

Reverse a String

Palindrome check

Sum of digits

Tower of Hanoi

Subsets generation

N-Queens problems

Tree traversal



Arrays and Strings in Java

⇒ Array in Java

→ Definition: - An array is a data structure that stores a fixed-size sequential collection of elements of the same type.

→ Why use Arrays?

Efficient memory usage

Easy element access via index

Suitable for static data checks

→ Syntax:

```
<datatype>[] arrayName = new <datatype> [size];
```

→ Example:

```
int[] arr = new int[5];
```

```
arr[0] = 10;
```

→ Initialization:

```
int[] arr = {1, 2, 3, 4, 5};
```

→ Traversal Example:

```
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

• Time Complexity:

Access: $O(1)$

Search: $O(n)$

Insertion / Deletion: $O(n)$

• Space Complexity: $O(n)$

⇒ String in Java

- ⇒ Definition:- A String is an object that represents a sequence of character
- ⇒ Why Strings are Special?
Immutable (once created, cannot be changed)
Stored in a special string pool
- ⇒ Declaration and Initialization:
String str = "Hello";
String str2 = new String ("World");
- ⇒ Concatenation:
String result = str + " " + str2
- ⇒ Common String Methods:
str.length()
str.charAt(i)
str.substring(start, end)
str.contains("abc")
str.equals("abc")
str.toCharArray()

Example Code:

```
public class StringExample {  
    public static void main (String [] args) {  
        String s = "Syntax Error";  
        System.out.println("Length: " + s.length());  
    }  
}
```


(a) Syntax Error

— Abhishek Rathor

Date: ___/___/___



```
System.out.println("char at 2: " + s.charAt(2));  
System.out.println("Substring(0,6): " + s.substring(0,6));  
}  
}
```

Time Complexity

Space Complexity: $O(n)$

Access: $O(1)$

Concatenation (naive): $O(n)$

Searching substring: $O(n \cdot m)$ where m is length of pattern

Common Interview Questions

Array:

Find max/min in array

Reverse an array

Find duplicate elements

Kadane's Algorithm (Max subarray sum)

String:

Reverse a String

Palindrome check

Anagram check

Longest substring without repeating characters

Example: Reverse a String

```
public class ReverseString {
```

```
    public static void main (String[] args) {
```

```
        String s = "hello";
```

```
        String reversed = "";
```

```
        for (int i = s.length() - 1; i >= 0; i--) {
```

```
            reversed += s.charAt(i);
```

```
        }
```


Date. / /

```
System.out.println("Reversed: " + reversed);  
}
```

Time Complexity: $O(n^2)$ (because of string concatenation)

Optimized: Use StringBuilder ($O(n)$)

Example: Reverse an Array

```
public class ReverseArray {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3, 4, 5};  
        int start = 0, end = arr.length - 1;  
        while (start < end) {  
            int temp = arr[start];  
            arr[start] = arr[end];  
            arr[end] = temp;  
            start++;  
            end--;  
        }  
        for (int num : arr) {  
            System.out.print(num + " ");  
        }  
    }  
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$



Arrays (1D and 2D), Operations

1D Arrays

⇒ Definition:- A 1D Array is a linear structure that holds a fixed number of values of a single data type.

⇒ Declaration in Java

`int[] arr = new int[5];` // declaration and memory allocation

`int[] arr = {1, 2, 3, 4, 5};` // direct initialization

⇒ Real-Life Example:-

Storing marks of 5 students: `int[] marks = {90, 85, 78};`

2D Arrays

⇒ Definition:- A 2D Array is an array (like a Matrix)

⇒ Declaration in Java:-

`int[][] matrix = new int[3][3];` // 3x3 Matrix

`int[][] matrix = {{1, 2}, {3, 4}, {5, 6}};` // Initialization

⇒ Example Use:- Used to represent tables, matrices on grids in games.

Array Operations

1. Insert:

Insert at index (manually shift elements);

java

```
for (int i = n; i > pos; i--) {
    arr[i] = arr[i-1];
}
```

arr[pos] = value;

2. Delete: Delete at index:

```
for (int i = pos; i < n-1; i++) {
    arr[i] = arr[i+1];
}
```

3. Update:

arr[Index] = newValue;

4. Prefix Sum

⇒ Definition:- Prefix Sum is technique to preprocess an array to answer range sum queries in $O(1)$

⇒ Example:

Original: [2, 3, 5, 1] ⇒ Prefix: [2, 5, 10, 11]

⇒ Code:

```
int[] prefix = new int[arr.length];
```

```
prefix[0] = arr[0];
```

```
for (int i = 1; i < arr.length; i++) {
    prefix[i] = prefix[i-1] + arr[i];
}
```




5. Sliding Window

⇒ Definition:- A technique to work on contiguous subarrays of a fixed or variable size, optimizing brute force solutions.

Use case: Max sum of subarray of size k .

⇒ Code:

```
int maxSum(int[] arr, int k) {
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += arr[i];
    int max = windowSum;
    for (int i = k; i < arr.length; i++) {
        windowSum += arr[i] - arr[i - k];
        max = Math.max(max, windowSum);
    }
    return max;
}
```

6. Kadane's Algorithm

Definition:- Used to find the maximum subarray sum in $O(n)$ time.

Key Idea:- Track currentSum, if it goes below 0, reset it.

Code:

```
int maxSubArray(int[] arr) {
    int max = arr[0], curr = arr[0];
    for (int i = 1; i < arr.length; i++) {
        curr = Math.max(arr[i], curr + arr[i]);
        max = Math.max(max, curr);
    }
    return max;
}
```


Sorting Algorithms (Java Code + Theory)

1. Bubble Sort

- ⇒ Theory :- Repeatedly swap adjacent elements if they are in the wrong order. Inefficient but easy.
- ⇒ Time : $O(n^2)$
- ⇒ Best : $O(n)$ (Already sorted)

2. Selection Sort

- ⇒ Theory : Find the minimum element and place it at the beginning. Repeat for the rest.
- ⇒ Time : $O(n^2)$
- ⇒ Space : $O(1)$

3. Insertion Sort

- ⇒ Theory : Take an element and Insert it into the correct position in the sorted left part.
- ⇒ Time : $O(n^2)$, Best : $O(n)$
- ⇒ Use Case : Small datasets

4. Merge Sort

Theory : Divide array into halves, sort each half, then merge. It's a divide and conquer algorithm.

Date. / /

⇒ Time: $O(n \log n)$

⇒ Space: $O(n)$

5. Quick Sort

⇒ Theory: Pick a pivot and partition the array such that $\text{left} < \text{pivot} < \text{right}$. Then sort both halves recursively.

⇒ Time:

6. Counting Sort

⇒ Theory:- Count frequency of elements and reconstructed sorted array. Works only for non-negative integers.

⇒ Time: $O(n+k)$

⇒ Space: $O(k)$ where $k = \text{max element}$

Summary Table

Algorithm	Time Complexity	Space	Stable	Use Case
Bubble Sort	$O(n^2)$	$O(1)$	Yes	Teaching, basic
Selection Sort	$O(n^2)$	$O(1)$	No	Memory-limited
Insertion Sort	$O(n^2)$	$O(1)$	Yes	Nearly Sorted Array
Merge Sort	$O(n \log n)$	$O(n)$	Yes	Large datasets
Quick Sort	$O(n \log n)$	$O(\log n)$	No	Performance critical apps
Counting sort	$O(n+k)$	$O(k)$	Yes	Bounded range arrays

Strings

⇒ Definition: A string is sequence of characters. In Java, strings are immutable (cannot be changed after creation).

⇒ Common Operations:

```
String s = "Hello";
int len = s.length(); // Length of string
char c = s.charAt(1); // Access character at index 1
String sub = s.substring(1, 4); // Substring from index 1 to 3
String upper = s.toUpperCase(); // "HELLO"
String concat = s + " world"; // "hello world"
```

Palindrome Check

⇒ Definition: A string is a palindrome if it reads the same forward and backward.

⇒ Example: "madam", "racecar"

⇒ Code:

```
boolean isPalindrome(String s) {
    int i = 0; j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i) != s.charAt(j)) return false;
        i++;
        j--;
    }
    return true;
}
```


Anagram check

⇒ Definition:- Two strings are anagrams if they contain the same characters in any order

⇒ Example: "listen" and "silent"

⇒ Code:

```
boolean isAnagram (String s1, String s2) {
    if (s1.length() != s2.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s1.length(); i++) {
        count[s1.charAt(i) - 'a']++;
        count[s2.charAt(i) - 'a']--;
    }
    for (int c : count) {
        if (c != 0) return false;
    }
    return true;
}
```

String Builder in Java

⇒ Definition:- StringBuilder is used for mutable string manipulation. It is more efficient than using + for concatenation

⇒ Operations:

```
StringBuilder sb = new StringBuilder("hello");
sb.append(" world");           // "hello world"
sb.reverse();                  // Reverse string
```


`sb.deleteCharAt(2); // Delete character at index 2`
`String result = sb.toString();`

Pattern Matching Algorithms

1. KMP (Knuth-Morris-Pratt)

- Idea: Preprocess the pattern to avoid redundant comparison
- Use Case: Efficient pattern searching in large text
- Time: $O(n+m)$ (n = length of text, m = length of pattern)

2. Rabin-Karp

- Idea:- Use rolling hash to compare substrings
- Use Case:- Multiple pattern matching, plagiarism detection
- Time: $O(n+m)$, worst $O(nm)$ with bad hash

Substrings

⇒ Definition:- A substring is a contiguous sequence of characters within a string

⇒ Count of all substrings:-

For a string of length n , number of substrings = $\frac{n*(n+1)}{2}$

Ⓢ Syntax Error

— Abhishek Rathor

Date. ___/___/___

⇒ Code to Print All substrings:

```
for (int i = 0 ; i < s.length() ; i++) {  
    for (int j = i+1 ; j <= s.length() ; j++) {  
        System.out.println(s.substring(i, j));  
    }  
}
```

Longest Common Prefix

⇒ Definition: The longest starting portion of all strings that is common

⇒ Use Case: Auto-suggestions, command-line tools.

⇒ Code:

```
String longestCommonPrefix (String[] strs) {  
    if (strs.length == 0) return "";  
    String prefix = strs[0];  
    for (int i = 1 ; i < strs.length ; i++) {  
        while (strs[i].indexOf(prefix) != 0) {  
            prefix = prefix.substring(0, prefix.length() - 1);  
            if (prefix.isEmpty()) return "";  
        }  
    }  
    return prefix;  
}
```


Date. / /

Summary Table

Topic	Use Case	Time-Complexity
• Palindrome Check	Validation, puzzles	$O(n)$
• Anagram Check	Word Comparison	$O(n)$
• String Builder	Efficient modification	$O(1)$ / $O(n)$
• KMP Algorithm	Pattern search	$O(n+m)$
• Rabin-Karp Algorithm	Plagiarism check	Avg: $O(n+m)$
• Substrings	Combinations in string	$O(n^2)$
• Longest Common Prefix	Auto-Complete	$O(n \cdot m)$



Searching And Binary Search

Linear Search

⇒ Definition:- Check each element one by one to find the target

⇒ Use Case:- When the array is unsorted or small

⇒ Code:

```
int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}
```

Important Table

Case	Time Complexity	Space Complexity	Explanation
Best Case	$O(1)$	$O(1)$	Element is found at the first index
Average Case	$O(n)$	$O(1)$	Element is found somewhere in the middle
Worst Case	$O(n)$	$O(1)$	Element is not present or found at the last index

Notes:

- n = number of element in the array
- Space Complexity is $O(1)$ because linear search uses no additional space, just a few variable for indexing or comparison.

Binary Search (on Sorted Array)

⇒ Definition: - Divide and conquer: check mid element and search in the appropriate half

⇒ Use case: - Efficient Search in Sorted Array

⇒ Code:

```
int binarySearch(int[] arr, int target) {
    int low = 0; high = arr.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Important Table:

Case	Time Complexity	Space Complexity (Recursive)	Space Complexity (Iterative)
Best Case	$O(1)$	$O(1)$	$O(1)$
Average Case	$O(\log n)$	$O(\log n)$	$O(1)$
Worst Case	$O(\log n)$	$O(\log n)$	$O(1)$



Notes:

- n = number of elements in the sorted array
- Iterative version uses $O(1)$ space because it only uses loop variables
- Recursive version uses $O(\log n)$ space due to the recursive call stack

Lower Bound

⇒ Definition:- The first index where the element is greater than or equal to the target.

⇒ Code:

```
int lowerbound (int [] arr, int target) {  
    int low = 0, high = arr.length;  
    while (low < high) {  
        int mid = (low + high) / 2;  
        if (arr[mid] < target) low = mid + 1;  
        else high = mid;  
    }  
    return low;  
}
```

⇒ Use case : Frequency and range queries

⇒ Time Complexity: $O(\log n)$

⇒ Array Type: Sorted



Upper Bound

⇒ Definition:- The first index where the element is greater than the target

⇒ Code:-

```
int upperBound (int[] arr, int target) {  
    int low = 0, high = arr.length;  
    while (low < high) {  
        int mid = (low + high) / 2;  
        if (arr[mid] <= target) low = mid + 1;  
        else high = mid;  
    }  
    return low;  
}
```

⇒ Use Case: Frequency or Range queries

⇒ Time Complexity: $O(\log n)$

⇒ Array Type: Sorted

Search in Rotated Sorted Array

⇒ Definition: Array is sorted but rotated. Find the index of the target efficiently.

⇒ Idea:- Find which half is sorted and then decide which part to discard



⇒ Code:

```
int searchRotated (int[] arr, int target) {
    int low = 0; high = arr.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) return mid;
        if (arr[low] <= arr[mid]) { // left part sorted
            if (target >= arr[low] && target < arr[mid])
                high = mid - 1;
            else low = mid + 1;
        } else { // Right part sorted
            if (target > arr[mid] && target <= arr[high])
                low = mid + 1;
            else high = mid - 1;
        }
    }
    return -1;
}
```

⇒ Time Complexity: $O(\log n)$

Binary Search on Answer Problems

⇒ Definition :- Apply binary search on a range of answers ~~int~~ instead of the array itself.

⇒ Use Case:

When you're optimizing or finding min/max value like.

- Min days to complete tasks
- Max pages read per day.
- Aggressive cows, Painters partition, etc.

⇒ Structure Template:

```
int binarySearchAnswer (int[] arr) {
    int low = min-possible, high = max-possible;
    int ans = -1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (isValid (mid)) {
            ans = mid;
            high = mid - 1; // or low = mid + 1
        } else {
            based on condition
            low = mid + 1; // or high = mid - 1
        }
    }
    return ans;
}
```

⇒ Important: Define the isValid() function based on the problem

SYNTAX ERROR // Abhishek Rather

These hand written notes are exclusively created for educational purpose under the SYNTAX ERROR brand

Unauthorised use, duplication or redistribution in any form is strictly prohibited for collaboration or queries : @code.abhi; 07

Recursion & Backtracking

⇒ What is Recursion?

- Definition: When a function calls itself to solve smaller sub-problems.

- Structure:

```
void function() {
    // base case
    // recursive call(s)
}
```

- Key Concepts:-

- Base Case: Stops the recursion
- Recursive Case: Breaks the problem into smaller pieces.

⇒ Factorial using Recursion

- Code:

```
int factorial(int n) {
    if (n == 0 || n == 1) return 1; // base case
    return n * factorial(n-1); // recursive call
}
```

- Time Complexity: $O(n)$

⇒ Fibonacci using Recursion

- Code:

```
int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n-1) + fibonacci(n-2);
}
```




- Time Complexity :- $O(2^n)$ → use memoization for optimization
- Note: Not efficient for large n

Subsets (Power Set)

⇒ Definition: Generate all possible subsets of given array strings

⇒ Code:

```
void generateSubsets(int[] arr, int index, List<
    List<Integer> current, List<List<Integer>
    result) {
```

```
    if(index == arr.length) {
```

```
        result.add(new ArrayList<>(current));
```

```
        return;
```

```
    }
```

```
    // include
```

```
    current.add(arr[index]);
```

```
    generateSubsets(arr, index + 1, current, result);
```

```
    // exclude
```

```
    current.remove(current.size() - 1);
```

```
    generateSubsets(arr, index + 1, current, result);
```

```
}
```

⇒ What is Backtracking?

⇒ Definition :- Try all possible options and backtrack (undo the choice) if a solution is not valid.

⇒ Use Case :- Solving constraint-based problems like puzzles, paths, arrangements.

Q) Syntax Error
— Abhishek Kathor
Date: ___/___/___



⇒ Steps:

- 1) Choice
- 2) Explore
- 3) Unchoose (Backtrack)

N-Queens Problem

⇒ Definition:- Place N queens on an $N \times N$ chessboard such that no two queens attack each other.

⇒ Code Snippet (Simplified):

```
boolean isSafe(char[][] board, int row, int col) {  
    // check column, left diagonal, right diagonal.  
    // .....
```

```
}
```

```
void solve(char[][] board, int row, List<List<String>>  
    result) {
```

```
    if (row == board.length) {
```

```
        List<String> solution = new ArrayList<>();
```

```
        for (char[] r : board)
```

```
            solution.add(new String(r));
```

```
        result.add(solution);
```

```
        return;
```

```
    }
```

```
    for (int col = 0; col < board.length; col++) {
```

```
        if (isSafe(board, row, col)) {
```

```
            board[row][col] = 'Q';
```

```
            solve(board, row + 1, result);
```

```
            board[row][col] = '.'; // backtrack
```

```
        }
```

```
    }
```

```
}
```




⇒ Time Complexity: $O(N!)$

Rat in Maze

⇒ Definition:- From top-left to bottom right, move in all directions to find all paths.

⇒ Allowed Moves: Down, Right, Up, Left.

⇒ Code Snippet:

```
void solveMaze(int[][] maze, int x, int y, int[][]  
    visited, String path) {  
    if (x == n-1 && y == n-1) {  
        result.add(path);  
        return;  
    }
```

// Down

```
    if (isValid(x+1, y)) {
```

```
        visited[x][y] = 1;
```

```
        solveMaze(maze, x+1, y, visited, path + "D");
```

```
        visited[x][y] = 0; // backtrack
```

```
    }
```

```
    // Same for right, up, left ...
```

```
}
```

⇒ Time Complexity: $O(4^{n^2})$ worst case

Sudoku Solver

⇒ Definition:- Fill a 9x9 board so that each row, column, and 3x3 grid has digits 1-9.

Date. —/—/—



⇒ Code Snippet

```
boolean isValid(char[][] board, int row, int col, char num) {  
    // check row, col, and 3x3 grid  
}  
  
boolean solve(char[][] board) {  
    for (int i = 0; i < 9; i++) {  
        for (int j = 0; j < 9; j++) {  
            if (board[i][j] == ".") {  
                for (char c = '1'; c <= '9'; c++) {  
                    if (isValid(board, i, j, c)) {  
                        board[i][j] = c;  
                        if (solve(board)) return true;  
                        board[i][j] = '.'; // backtrack  
                    }  
                }  
            }  
        }  
        return false;  
    }  
    return true;  
}
```

⇒ Time Complexity : $O(9^8 \cdot 81)$ worst case

Summary Table

Problems	Type	Technique	Time Complexity
Factorial	Recursion	Base case + call	$O(n)$
Fibonacci	Recursion	Tree Recursion	$O(2^n n)$
Subsets	Recursion	Include/Exclude	$O(2^n n)$
N-Queens	Backtracking	Safe placing	$O(N!)$
Rat in a Maze	Backtracking	Move in 4 dirs	$O(4^n (n^2))$
Sudoku Solver	Backtracking	Fill + Undo	$O(9^{81})$



HASHING

⇒ Definition: Hashing maps data (keys) to a fixed size value (index) using a hash function.
It allows $O(1)$ average time complexity for insertion, deletion, and lookup.

⇒ HashMap and HashSet in Java

- `HashMap<K, V>` → Stores key-value pairs
- `HashSet<E>` → Stores only unique values (like keys of HashMap)

• HashMap Syntax:

```
HashMap<String, Integer> map = new HashMap<>();  
map.put("apple", 3);  
map.get("apple"); // return 3  
map.containsKey("apple"); // true  
map.remove("apple");
```

• HashSet Syntax:

```
HashSet<Integer> set = new HashSet<>();  
set.add(5);  
set.contains(5); // true  
set.remove(5);
```

Frequency Counting using HashMap

- Problem :- Count how many times each element appears
- Use Case :- Count occurrences

Code:

```
int[] arr = {1, 2, 2, 3, 3, 3};
```

```
HashMap<Integer, Integer> freq = new HashMap<>();
```

```
for (int num : arr) {
```

```
    freq.put(num, freq.getOrDefault(num, 0) + 1);
}
```

• Output : { 1=1, 2=2, 3=3 }

• Time Complexity : $O(n)$

Two Sum Problem

• Problem: Find indices of two numbers that add up to a target

• Input: arr = [2, 7, 11, 15], target = 9

• Output: (0, 1) → because $2 + 7 = 9$

Code:

```
public int[] twoSum(int[] nums, int target) {
```

```
    HashMap<Integer, Integer> map = new HashMap<>();
```

```
    for (int i = 0; i < nums.length; i++) {
```

```
        int diff = target - nums[i];
```

```
        if (map.containsKey(diff)) {
```

```
            return new int[] {map.get(diff), i};
```

```
        }
```

```
        map.put(nums[i], i);
```

```
    }
```

```
    return new int[] {-1, -1};
```

```
}
```

• Time Complexity : $O(n)$

• Use Case :- pair sum equals target



Subarrays with Given Sum (Positive Integers Only)

- Problem: Count subarrays with a specific sum (e.g. 5)

- Code :

```
public int subarraySum(int[] nums, int k) {  
    HashMap<Integer, Integer> map = new HashMap<>();  
    map.put(0, 1); // base case  
    int sum = 0; count = 0;  
    for (int num : nums) {  
        sum += num;  
        count += map.getOrDefault(sum - k, 0);  
        map.put(sum, map.getOrDefault(sum, 0) + 1);  
    }  
    return count;  
}
```

- Time Complexity : $O(n)$
- Technique : prefix sum + HashMap

Interview Level Hash-Map Problems

1) Long



Interview - Level HashMap Problems

Longest Consecutive Sequence

- Problem: Find the length of the longest consecutive elements sequence

• Code:

```
public int longestConsecutive(int[] nums) {  
    HashSet<Integer> set = new HashSet<>();  
    for (int num : nums) set.add(num);  
    int maxlen = 0;  
    for (int num : set) {  
        if (!set.contains(num - 1)) { // start of sequence  
            int curr = num;  
            int len = 1;  
            while (set.contains(curr + 1)) {  
                curr++;  
                len++;  
            }  
            maxlen = Math.max(maxlen, len);  
        }  
    }  
    return maxlen;  
}
```

- Time Complexity: $O(n)$
- Use case :- Continuous elements



2. Group Anagrams

- Problem: Group strings that are anagrams

- Code:

```
public List<List<String>> groupAnagrams(String[] strs) {
    HashMap<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] ch = s.toCharArray();
        Arrays.sort(ch);
        String key = new String(ch);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}
```

- Time Complexity: $O(n * k \log k)$
(n = no. of words, k = avg length of word)
- Use Case - Anagram grouping



Stack & Queue

Stack

Definition :- A stack is a linear data structure that follows LIFO (Last-In-First-Out) principle. Stack has one end, whereas queue has two ends (front and rear).

A stack is a container in which insertion & deletion can be done from the end known as the top of the stack.

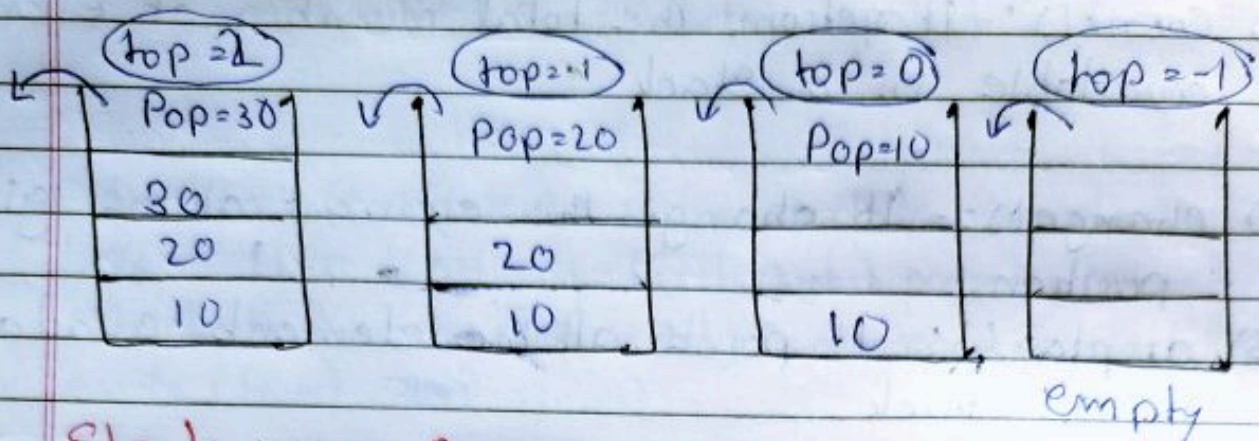
A stack is an Abstract Data Type with a pre-defined capacity, which means that it can store elements of limited size.

Operations on the Stack :-

- 1) **Push ()**: When we insert an element in a stack then the operation is known as push. If stack is full overflow condition occurs.
- 2) **Pop ()**: When we delete an element from stack, the operation is called as pop (). If stack is empty means no element exists in the stack, this state is known as underflow state.
- 3) **Peek ()** :- It returns the element at given position.

Date. _/_/

rehtar joshin



Stack using Array

```
class MyStack {
```

```
    int[] stack;
```

```
    int top;
```

```
    MyStack(int size) {
```

```
        stack = new int[size];
```

```
        top = -1;
```

```
    }
```

```
    void push(int val) {
```

```
        if (top == stack.length - 1) return; // overflow
```

```
        stack[++top] = val;
```

```
    }
```

```
    int pop() {
```

```
        if (top == -1) return -1; // underflow
```

```
        return stack[top--];
```

```
    }
```

```
    int peek() {
```

```
        return (top == -1) ? -1 : stack[top];
```

```
    }
```

```
}
```


Date. —/—/—



Stack using Linked List

```
class Node{
    int data;
    Node next;
    Node (int data) {this.data = data;}
}

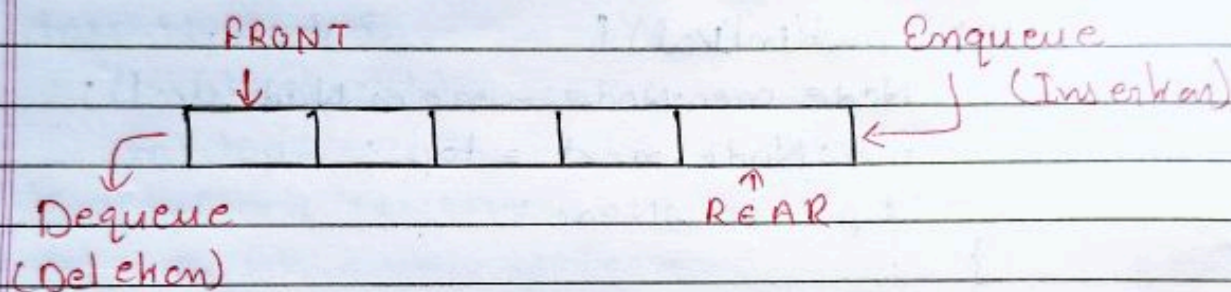
class StackLL{
    Node top = null;
    void push(int val) {
        Node newNode = new Node(val);
        newNode.next = top;
        top = newNode;
    }
    int pop() {
        if (top == null) return -1;
        int val = top.data;
        top = top.next;
        return val;
    }
    int peek() {
        return (top == null) ? -1 : top.data;
    }
}
```




Queue

A queue can be defined as ordered list which enables insert operations to be performed at at one end called REAR and delete operations to be performed at another end called FRONT.

→ Queue can be referred as to be first In first out list (FIFO).



Complexity of Queue :-

	Average				Space Comp Worst
	Access	Search	Deletion	Insertion	
Queues	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Queue	Worst				
	Access	Search	Insertion	Deletion	
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	

Operation on Queue :-

1) Enqueue :- Enqueue is used to insert element at rear end of the queue. It return void



2. Dequeue:- dequeue operations performs the deletion from front end of queue. The deque operation can also be designed to void it.
3. Peek: This returns element which is pointed by front pointer in the queue but does not delete it.
- 4) Queue overflow (is Full):- When queue is completely full then it shows overflow condition
- 5) Queue underflow (is Empty):- When there is no element in the queue then it shows underflow condition.

Types of Queue:-

- ① Linear queue:- In linear queue, an insertion takes place from one end while deletion takes place from another end. It strictly follows FIFO Rule. The linear queue can be represented, as shown:-

Front $\rightarrow$

10	20	30	40	50
----	----	----	----	----

 $\leftarrow$ Rear

- Elements are added at the rear end.
- Elements are removed from the front end.
- When we keep adding elements, the rear moves forward.

Date. ___/___/___



Drawback of Linear Queue:-

- Insertion can only happen at the rear.
- Once the queue is full (rear reaches the last index), new elements cannot be added, even if there are empty spots at the front due to deletion.
- This leads to a waste of space and is called the overflow condition, even though the queue might not be truly full.

2) Circular Queue :- In a circular queue, the last position is connected back to the first position, forming a circle.

- It works like a linear queue but makes better use of space.
- When the rear reaches the end, it goes back to the beginning if there's space.
- This helps to reuse the freed space at the front after deletion.



Drawback of Linear queue is overcome in circular queue. If empty space is available, new element can be added to empty space by simply incrementing value of rear .

3. Priority queue:- The queue in which each element has some priority associated with it based on priority of the elements, elements are arranged in a priority queue. If elements occurs with same priority, then they are served according to FIFO principle.

Queue Implementation using Array.

```
class MyQueue {
    int[] queue;
    int front = 0, rear = 0;
    MyQueue(int size) {
        queue = new int[size];
    }
    void enqueue(int val) {
        if (rear == queue.length) return; // overflow
        queue[rear++] = val;
    }
    int dequeue() {
        if (front == rear) return -1;
        return queue[front++];
    }
    int peek() {
        return (front == rear) ? -1 : queue[front];
    }
}
```


Date. —/—/—

Queue using Linked list

```
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

class QueueLL {
    Node front = null, rear = NULL;
    void enqueue(int val) {
        Node newNode = new Node(val);
        if (rear == NULL) front = rear = newNode;
        else {
            rear.next = newNode;
            rear = newNode;
        }
    }

    int dequeue() {
        if (front == null) return -1;
        int val = front.data;
        front = front.next;
        if (front == null) rear = null;
        return val;
    }
}
```

Inbuilt Stack & Queue in Java

```
Stack<Integer> stack = new Stack<>();
stack.push(10); stack.pop(); stack.peek();
```


Date. — / — / —



```
Queue<Integer> queue = new LinkedList<>();  
queue.offer(5); queue.poll(); queue.peek();
```

Prefix, Infix, Postfix Expressions

- Infix: $A + B \rightarrow$ Human readable
- Postfix: $AB + \rightarrow$ Evaluated using stack
- Prefix: $+AB \rightarrow$ Also evaluated using stack

Evaluate Postfix Expression

```
public int evalPostfix (String[] tokens) {  
    Stack<Integer> stack = new Stack<>();  
    for (String token : tokens) {  
        if (" + - * / ".contains(token)) {  
            int b = stack.pop();  
            int a = stack.pop();  
            switch (token) {  
                case "+": stack.push(a+b); break;  
                case "-": stack.push(a-b); break;  
                case "*": stack.push(a*b); break;  
                case "/": stack.push(a/b); break;  
            }  
        } else {  
            stack.push(Integer.parseInt(token));  
        }  
    }  
    return stack.pop();  
}
```

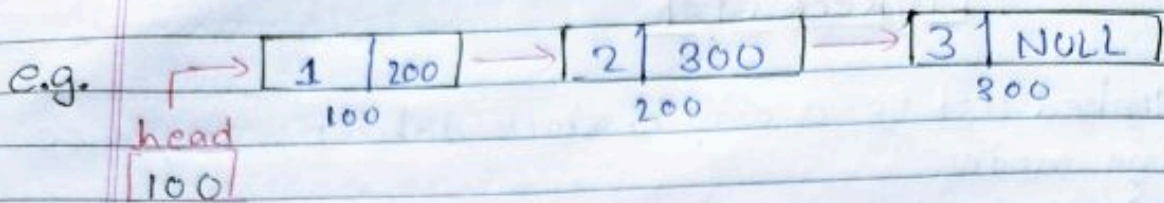

Date. —/—/—



Do it yourself

Monotonic Stack / Queue Famous Problems.

- Next Greater Element
- Next Smaller Element
- Sliding window Maximum (Variant)
- Histogram problems.



⇒ Example:

```

class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = NULL;
    }
}
  
```

⇒ Explanation: (see me above. Example)

Node head = new Node(10);

head.next = new Node(20);

head.next.next = new Node(30);

Each node stores the value and reference to the next node. Traversing from head gives all elements

2. Doubly Linked List :-

• Each node has three parts:

a. prev - link to the previous node.

b. data - value

c. next - link to the next node

• You can move both forward and backward



⇒ Example:

```
class Node {
    int data;
    Node prev, next;
    Node(int data) {
        this.data = data;
    }
}
```

⇒ Explanation:-

Node First = new Node(10);

Node Second = new Node(20);

Node Third = new Node(30);

First.next = Second;

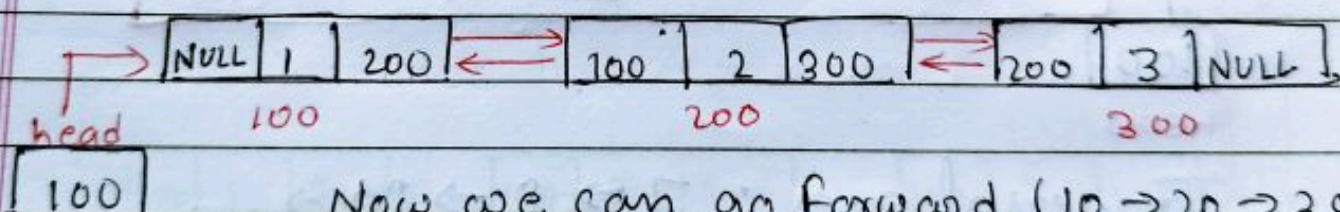
Second.prev = First;

Second.next = Third;

Third.prev = Second;

ⓐ SYNTAX ERROR

- Abhishek Kattor



Now we can go forward (10 → 20 → 30)
or backward (30 → 20 → 10)

3. Circular Linked List:-

ⓐ Circular Singly Linked List:

In a circular Singly linked list, the last node points to the first node, instead of NULL, forming a loop.

Date. —/—/—



⇒ Example:-

```
class Node {  
    int data;  
    Node next;  
    Node prev;  
    Node (int data) { this.data = data; }  
}
```

Node head = new Node (1);

Node Second = new Node (2);

Node Third = new Node (3);

head.next = Second; // connect forward

Second.next = Third;

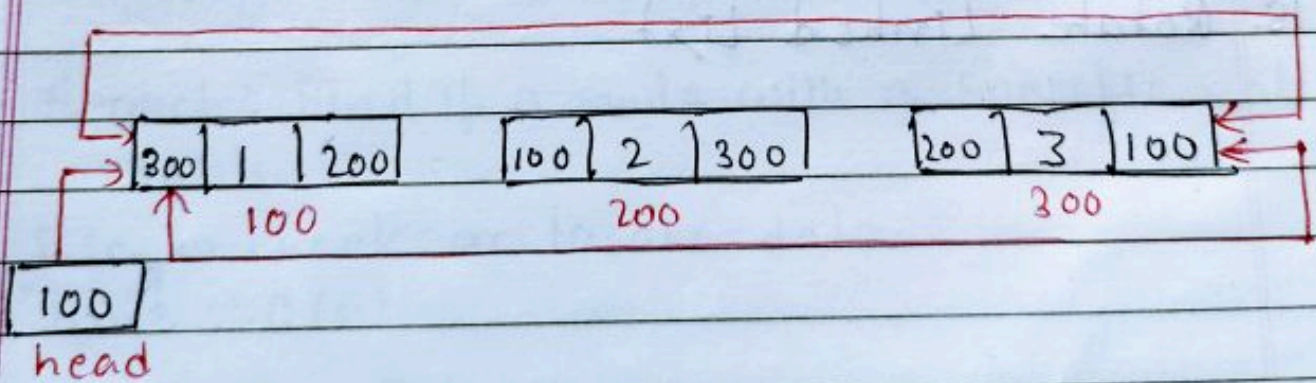
Third.next = head;

// Circular connection (last to first)

head.prev = Third; // connect backward

Second.prev = head;

Third.prev = Second;



Date. — / — / —

Important (Must to do)

Most Famous And Commonly Asked Linked List interview questions.

1. Reverse a Linked List
2. Find the middle of the Linked List
3. Detect a cycle in Linked List
4. Length of a Linked List
5. Check if a Linked List is a Palindrome
6. Remove Duplicate from a Sorted Linked List
7. Merge Two Sorted Linked List
8. Remove Nth Node from End of List
9. Intersection Point of Two Linked Lists
10. Cycle Length and Starting Node
11. Delete a Node without head pointer
12. Clone a Linked List with Random pointers
13. LRU Cache (using Linked List + Hash Map)
14. Reverse Nodes in K-group
15. Rotate Linked List



Operations of linked list.

1. Traversal :- Visiting each node in the list from start to end.
 - Use to display or access all data.
 - Time : $O(n)$
2. Insertion :- Adding a new node to the list.
 - At beginning : Insert node before the head. $O(1)$
 - At end : Add node after the last node. $O(n)$
 - At position : Insert node at specific place.
3. Deletion :- Removing a node from the list.
 - From beginning : Remove the head : $O(1)$
 - From End : Remove the last node. $O(n)$
 - From position : Remove node from a specific place. $O(n)$
4. Search : Find if a node with a specific value exists.
Use to check or locate data
Time : $O(n)$
5. Update :- Change the value of a node at a given position.
Time : $O(n)$

Date: / /



6. Reverse :- Reverses the direction of the list.

Time : $O(n)$

Space : $O(1)$

7. Length :- Counts the number of nodes in the list
Used to know the size

Time : $O(n)$

SYNTAX ERROR || Abhishek Rathor

These handwritten notes are exclusively created for educational purposes under the SYNTAX ERROR brand

Unauthorized use, duplication, or redistribution in any form is strictly prohibited

for collaboration or Queries - (a) code.abhi07



Tree

We read data structure, like an array, linked list, stack and queue in which all elements are arranged in a sequential manner.

A tree is a special type of data structure where data is stored in a hierarchical (top to bottom) form.

It starts from root node

Every node can have child nodes

A node with no child is called a leaf

Basic Terminology

1. Root : Topmost node (starting point)
2. Parent : Node that has children
3. Child : Coming from parent
4. Leaf/External Node : Node with no children
5. Internal Node : Node with at least one child.
6. Siblings : Nodes with the same parent
7. Degree : Number of children a node has
8. Level : Distance from root (root is level 0)
9. Height/Depth : Longest path from root to leaf.
10. Ancestors :- Nodes above a given node in the tree.
Like your parents, grandparents.

The root is always an ancestor of all other nodes

11. Descendants :- Nodes below a given node in the tree.
Like your children and grandchildren etc.
All the nodes connected under your node are descendants

Date. —/—/—

rebat x/aidA



Just Remember:

- Ancestors are above you
- Descendants are below you

Types of Tree Data Structure :-

1. Binary Tree
Each node can have at most 2 children (left and right)
2. Full Binary Tree
Every node has 0 or 2 children only
3. Perfect Binary Tree
All internal nodes have 2 children, and all leaf nodes are at the same level
4. Complete Binary Tree
All levels are completely filled except maybe the last, which is filled left to right.
5. Skewed Binary Tree
Nodes only have one child.
 - Left Skewed: Nodes have only left child
 - Right Skewed: Nodes have only right child

1. Binary Tree (BT) :- Each node has at most two children - left and right.

Basic Node Structure

```
class Node {
```

```
    int data;
```

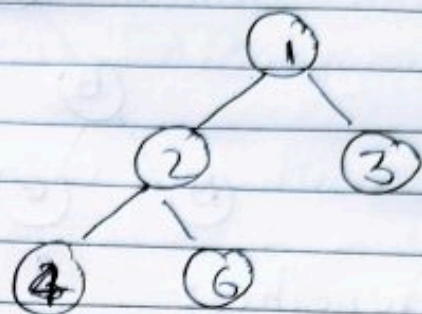
```
    Node left, right;
```

```
    Node(int data) {
```

```
        this.data = data;
```

```
    }
```

```
}
```

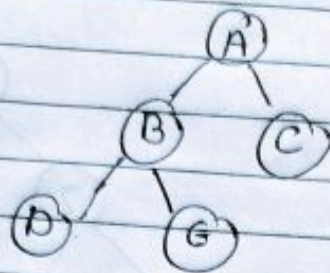


Properties of Binary Tree :-

- At each level of i , the maximum number of nodes is 2^i .
- The height of tree is longest path from root node to leaf node. In general, maximum number of nodes possible at height is $(2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^h)$.
- The minimum number of nodes possible at height h is equal to $h+1$.
- If number of nodes is minimum, then height of tree would be maximum.
- Minimum height can be computed as:
 $h = \log_2(n+1) - 1$
- Maximum height can be computed as:
 $h = n - 1$



- 2) Full / Proper / strict Binary Tree:-
Every node has 0 or 2 children.



Properties:-

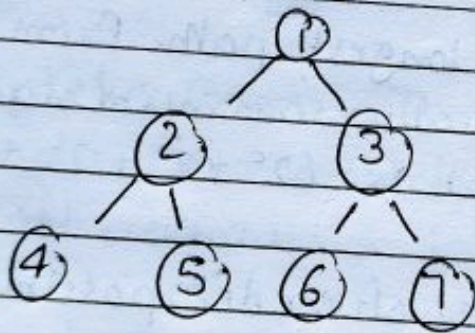
Maximum number of nodes : $2^{h+1} - 1$

Minimum number of nodes : $2^h - 1$

Minimum height $\log_2(n+1) - 1$

Maximum height $h = \frac{n+1}{2}$

3. Perfect Binary Tree:-



Note:- All the perfect Binary Trees are complete binary trees as well as the full Binary tree as But, vice versa is not true, all complete binary trees are full binary trees are the perfect Binary Trees

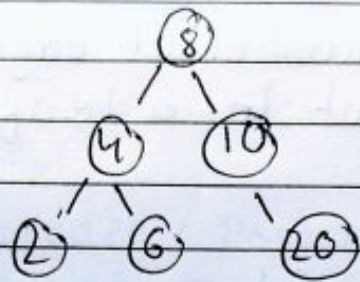
Date. ___/___/___



Binary Search Tree (BST):-

A special Binary tree where:

- Left child has smaller value than node
- Right child has greater value than node



@SYNTAX ERROR

— Abhishek Kothari

Key Operations:-

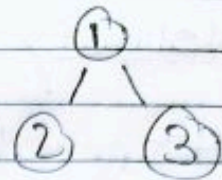
- Insert: Recursively add elements in correct order
- Search: Start at root, compare, move left or right
- Delete: Handle 0, 1 or 2 children cases

Tree traversals:- Ways to visit all nodes in a tree:

- Inorder (Left $\rightarrow$ Root $\rightarrow$ Right)
Prints nodes in increasing order in BST
- Preorder (Root $\rightarrow$ Left $\rightarrow$ Right)
Used to create a copy of the tree
- Postorder (Left $\rightarrow$ Right $\rightarrow$ Root)
Used to delete the tree



- Inorder: 2 1 3
- Preorder: 1 2 3
- Postorder: 2 3 1



4. Height of Tree

Definition:- The number of edges on the longest path from the root to a leaf.

Code:

```

int height(Node root) {
    if (root == NULL) return 0;
    return 1 + Math.max(height(root.left),
                        height(root.right));
}
  
```

5. Diameter of Tree :- The longest path between any two nodes (not necessarily through the root)

Formula:-

$$\text{diameter} = \max(\text{left height} + \text{right height} + 1)$$

6. Views of Binary Trees

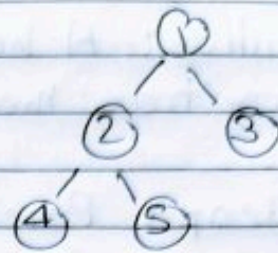
- Top View :- Nodes visible from the top when tree is viewed horizontally.
- Bottom View :- Nodes visible from the bottom
- Left View :- Leftmost node at every level
- Right View :- Rightmost node at every level

Date. — / — / —



7. Zigzag Traversal :- Level order but alternate levels are reversed.

Zigzag :- 1, 3, 2, 4, 5



🌳 Top 10 Tree Interview Questions.

1. Inorder / Preorder / Postorder
2. Check if a Binary Tree is a BST
3. Lowest Common Ancestor (LCA)
4. Height / Depth of a tree
5. Diameter of a Binary tree
6. Level Order Traversal (BFS)
7. Invert a Binary Tree (Mirror Tree)
8. Serialize and Deserialize a Binary Tree
9. Path Sum
10. Convert Sorted Array to Balanced BST

Date. ___/___/___



Heap & Priority Queue

⇒ Definition:- A heap is a special complete Binary tree that satisfies the heap property

- Min Heap: Parent node $\leq$ child node
- Max-Heap: Parent node $\geq$ child node

⇒ Properties:-

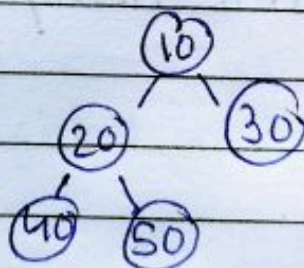
- Always Complete (all levels filled except possibly the last).
- Stored in an array (index-based logic)

⇒ Types of Heaps:-

- Min Heap:

Root contains the minimum element

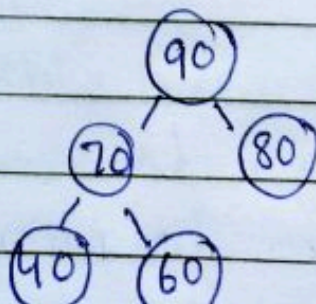
Example



- Max Heap:-

Root contains the maximum element

Example





⇒ Heap Operations

Operation	Time Complexity
Insert	$O(\log N)$
Delete (root)	$O(\log N)$
Peek (min/max)	$O(1)$

⇒ Java Priority Queue

Java provides Priority Queue in **java.util** package

- Default: Min-heap based on natural ordering

- Create Min-Heap:

```
PriorityQueue < Integer > minHeap = new PriorityQueue  
<>();
```

- Create Max-Heap:

```
PriorityQueue < Integer > maxHeap = new  
PriorityQueue <> (Collections.reverseOrder());
```

- Operations

```
minHeap.add(10);           // Insert  
minHeap.peek();           // Minimum element  
minHeap.poll();           // Remove minimum
```

⇒ Heapify (Build Heap)

- Heapify:

- Converts an array into a heap in $O(N)$ time
- Ensures the heap property is maintained from bottom-up



- Applications
Used in heap sort
Creating priority queues

⇒ Heap Sort

Steps:

1. Convert array to max-heap
2. Swap root (max) with last element
3. Reduce size and re-heapify
4. Repeat

- Time Complexity: $O(N \log N)$
- Space Complexity: $O(1)$ (in-place)

⇒ K Largest / Smallest Elements

- K Largest

Use Min-Heap of size k

Keep top k largest seen so far

- K Smallest

Use Max-Heap of size k

- Time Complexity: $O(N \log N)$

⇒ Median in a Stream

- Problem :-

Given a stream of integers, return the median after every new element

(a) Syntax Error
— Abhishek Rathor
Date: ___/___/___



• Approach:

Use 2 heaps:

Max-Heap (for lower half)

Min-Heap (for upper half)

Balance both heaps

Median:

If even count $\rightarrow$ average of tops

If odd count $\rightarrow$ top of larger heap

Example: Input: [5, 15, 1, 3]

Output: 5, 10, 5, 4

$\Rightarrow$ Summar Table:

Topic	Heap Used	Time Complexity
Insert/Delete	Min or Max	$O(\log N)$
Heapify	Min or Max	$O(N)$
Heap Sort	Max Heap	$O(N \log N)$
K Largest/Smallest	Min/Max Heap	$O(N \log N)$
Median In Stream	2 Heaps	$O(\log N)$ per insert

Date. / /

recd. reply 20
initial binding



Greedy Algorithms

⇒ A greedy algorithm is an approach that builds up an solution piece by piece, always choosing the locally optimal option at each step with the hope of finding a global optimum.

⇒ Key Characteristics:

- Make decisions greedily at every step
- Does not revisit earlier decisions
- Works only for problems with greedy property and optimal substructure.

⇒ General Approach for Greedy Problems:

1. Sort data (e.g., by end time, profit, rate)
2. Apply a greedy choice strategy
3. Make decisions step-by-step

1. Activity Selection Problem

Problem:- Given N activities with start and end times, select the maximum number of activities that can be performed by a single person, assuming that a person can only work on a single activity at a time.

Date. / /



⇒ Greedy Strategy:

Always pick the activity that ends earliest.

⇒ Time Complexity: $O(N \log N)$ (due to sorting)

⇒ Java Example:

```
import java.util.*;
```

```
class Activity {
```

```
    int start, end;
```

```
    Activity (int s, int e) {
```

```
        start = s;
```

```
        end = e;
```

```
    }
```

```
}
```

```
class ActivitySelection {
```

```
    public static void main (String [] args) {
```

```
        Activity[] activities = {
```

```
            new Activity (1, 3),
```

```
            new Activity (2, 4),
```

```
            new Activity (3, 5),
```

```
            new Activity (6, 7),
```

```
            new Activity (8, 9),
```

```
            new Activity (0, 6).
```

```
        };
```

```
        Arrays.sort (activities, Comparator.comparingInt  
            (a -> a.end));
```

```
        int count = 1;
```

```
        int lastEnd = activities [0].end;
```


Date. / /



```
for (int i = 1; i < activities.length; i++) {  
    if (activities[i].start >= lastEnd) {  
        count++;  
        lastEnd = activities[i].end;  
    }  
}  
System.out.println("Max activities:" + count);  
}
```

2. Huffman Coding

⇒ Problem:- Given characters and their frequencies, build an optimal prefix code (no code is prefix of another) with minimum average length

⇒ Greedy Strategy:

- Use a Min Heap to combine lowest frequency nodes
- Build a binary tree and assign codes from top to bottom

⇒ Time Complexity: $O(N \log N)$

⇒ Applications:

- File compression (ZIP, JPEG)
- Data encoding

3. Job Scheduling Problem

⇒ Problem:- Given N jobs with deadlines and profits, schedule jobs to maximize total profit such that only one job can be scheduled at a time and within its deadline.

⇒ Greedy Strategy:

- Sort jobs by profit in descending order.
- Schedule each job in the latest possible available slot before its deadline.

⇒ Time Complexity: $O(N \log N + N^2)$

⇒ Java Skeleton:

```
class Job {  
    int deadline, profit;  
    Job(int d, int p) { deadline = d; profit = p; }  
}
```

Questions Must To Do...

1. Fractional Knapsack
2. Minimum Platforms Problems



Trie's

⇒ A Trie (pronounced as "try") is a tree-based data structure used for storing strings in an efficient way, especially when the dataset contains many strings with common prefixes.

⇒ Key Features:

- Every node represents a character
- Paths from root to leaf represent words
- Used for prefix-based problems

⇒ Applications of Trie:

- Dictionary word storage
- Autocomplete
- Spell checker
- Prefix search
- Word segmentation (word break problem)

⇒ Trie Node Structure in Java

```
class TrieNode {
```

```
    TrieNode[] children = new TrieNode[26]; // a to z  
    boolean isEndOfWord;
```

```
    TrieNode() {
```

```
        isEndOfWord = false;
```

```
    }
```

```
}
```


Date. / /



1. Insert a Word into Trie

- ⇒ Idea: For each character in the word:
- Move to the corresponding child node
 - Create a node if it doesn't exist.

⇒ Time Complexity: $O(L)$ where L = length of word

⇒ Java Code:

```
void insert(String word) {
    TrieNode node = root;
    for (char ch : word.toCharArray()) {
        int index = ch - 'a';
        if (node.children[index] == null) {
            node.children[index] = new TrieNode();
        }
        node = node.children[index];
    }
    node.isEndOfWord = true;
}
```

2. Search a Word in Trie

- ⇒ Idea: Traverse the trie following each character in the word. If a required node is missing → return false.

⇒ Java Code:

```
boolean search(String word) {
    TrieNode node = root;
    for (char ch : word.toCharArray()) {
        int index = ch - 'a';
        if (node.children[index] == null) return false;
        node = node.children[index];
    }
    return node.isEndOfWord;
}
```




3. Prefix check (starts with)

⇒ Idea:- Same as search, but don't check for `isEndofWord`.

⇒ Time Complexity:- $O(L)$

⇒ Java Code:

```
boolean startsWith (String prefix) {  
    TrieNode node = root;  
    for (char ch : prefix.toCharArray()) {  
        int index = ch - 'a';  
        if (node.children[index] == null)  
            return false;  
        node = node.children[index];  
    }  
    return true;  
}
```

4. Word Break Problem

⇒ Problem: Given a string `s` and a dictionary of words, determine if `s` can be segmented into a space-separated sequence of dictionary words.

⇒ Approach:

Use Trie + Recursion + Memoization

⇒ Time Complexity:

$O(N \times L)$ where N = length of string,
 L = length of longest word



5. Longest Word with All Prefixes

→ Problem: Find the longest word in the dictionary such that all prefixes of the word are also in the dictionary.

→ Example:

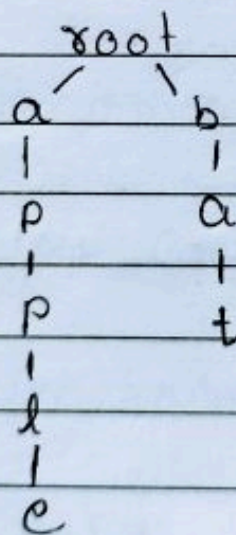
Input :- ["w", "wo", "wor", "worl", "world"]

Output :- "World"

→ Idea: Use DFS on Tree and only go to child nodes that are marked as **isEndOfWord = true**

🌳 Trie Tree Visual Example.

Let's say we insert: "apple", "app", "bat"



@SYNTAX-ERROR

- Abhishek Rathor

Date. / /



Summary Table

Operation	Time Complexity	Notes
• Insert	$O(L)$	L = length of word
• Search	$O(L)$	Fast lookup
• Prefix Search	$O(L)$	Useful for autocomplete
• Word Break	$O(N \times L)$	Use Trie + DP
• Longest Word	$O(N \times L)$	DFS + all prefixes exist check

Q) SYNTAX ERROR

- Not a valid identifier



Example: Fibonacci Numbers

1. Memoization (Top-Down)

```
static int fib(int n, int[] dp) {
    if (n <= 1) return n;
    if (dp[n] != -1) return dp[n];
    return dp[n] = fib(n-1, dp) + fib(n-2, dp);
}
```

2. Tabulation (Bottom-Up)

```
static int fib(int n) {
    int[] dp = new int[n+1];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

⇒ 0/1 Knapsack

Given: weights and values of n items, max weight W

Goal: Maximize value with 0 or 1 copy of copy item

DP Formula:

if ($wt[i] \leq W$)

$dp[i][W] = \max(dp[i-1][W], val[i] + dp[i-1][W - wt[i]]);$

Time Complexity: $O(n \times W)$

Space Complexity: $O(n \times W)$



⇒ Unbounded Knapsack
You can take unlimited instances of an item

Formula:

if $wt[i] \leq W$

$$dp[i][W] = \max(dp[i-1][W], val[i] + dp[i][W - wt[i]])$$

⇒ Subset Sum Problem
check: Is there a subset with sum = k?

Boolean DP:

$$dp[i][j] = dp[i-1][j] \parallel dp[i-1][j - arr[i]];$$

⇒ Partition Equal Subset Sum
Can we partition array into two subsets with equal sum?

Reduce to subset sum with $sum/2$.

⇒ Longest Common Subsequence (LCS)
Given: Two strings S_1, S_2
Goal: Length of longest common subsequence

Formula:

if $(S_1[i-1] == S_2[j-1])$

$$dp[i][j] = 1 + dp[i-1][j-1];$$

else

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1]);$$